perfetto-sql-py工具

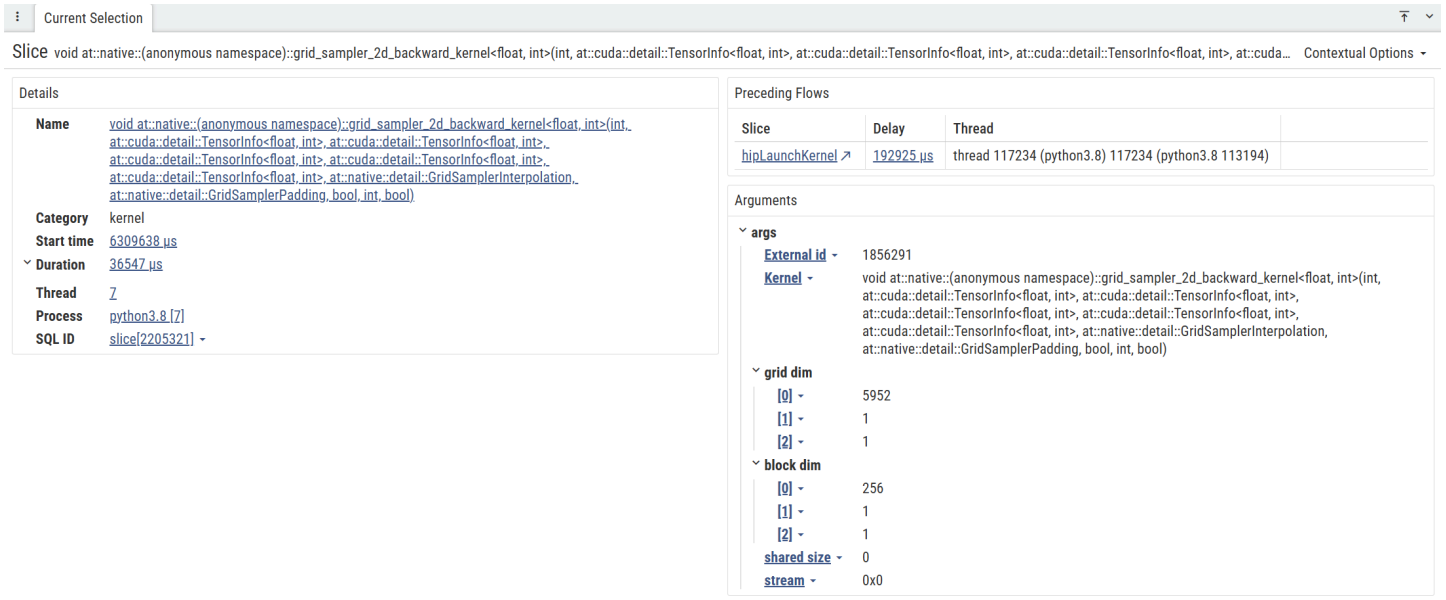
🦄 存储耗时、网格、shape等信息的json文件，使用perfetto打开之后，可以进行进一步分析，获取到一些详细信息。
当需要过滤字段的时候，也可以使用sql语句进行过滤（timeline也可以完成等价操作）

1. 详细信息获取

🎁 前提是使用torch的profiler工具，且记录了stack 和 shape

1.1 Kernel

在dcu端调度的kernel



1.1.1 基础信息

name：kernel名称

category：分类（kernel、runtime、cpu_op等分类）

starttime：开始的绝对时间（！！！需要注意：这个时间是绝对时间，在prof打开之后时间轴上会归零——这里后面sql查询会提及）

duration：运行时长

thread：线程

process：父进程

Sql ID：**唯一**的sql标志

1.1.2 维度信息

grid dim：kernel的网格大小（有3个维度，但是每个维度对应的是cpu_op中的那个维度这块不太清晰，【2】对应的是batch）

Block dim：block中线程大小

2. 筛选查询

此处以sql命令作为查询手段，可以扩展学习一下sql语句

2.1 常见命令

2.1.1 搜单一kernel

以kernel为案例，在json中有一个唯一的sql id

代码块

```
select * from slice where id=2205321
```

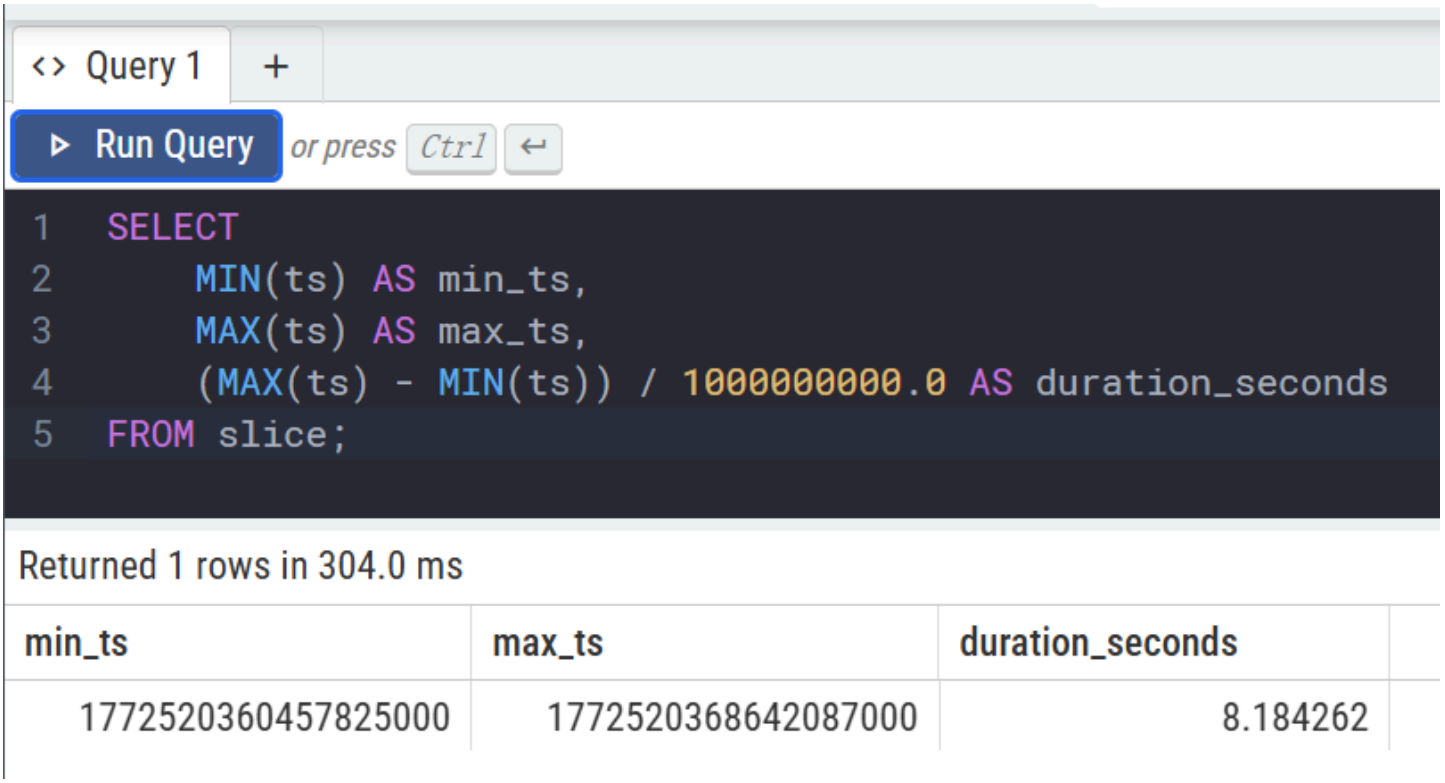
2.1.2 查询绝对时间

prof的json文件打开之后，时间轴是0-x，但是这个是归零之后的时间，其实在json上的开始时间是系统时间。

代码块

```
# prof的开始绝对时间和结束的绝对时间（这个是按照系统时间来的，用perfetto打开的话时间会归零）
SELECT
    MIN(ts) AS min_ts,
    MAX(ts) AS max_ts,
    (MAX(ts) - MIN(ts)) / 1000000000.0 AS duration_seconds
FROM slice;
```

搜索后显示开始时间和结束时间（ns为单位），以及持续时长



2.1.3 查询所有kernel耗时

代码块

```
SELECT
    name,
    COUNT(*) AS record_count,
    SUM(dur) / 1000.0 AS total_duration_us,
    SUM(dur) / COUNT(*) / 1000.0 AS avg_duration_us
FROM slice, trace_bounds  -- 1. 关联 trace_bounds 表获取起始时间
WHERE
    category IN('kernel', 'gpu_memcpy', 'gpu_memset')
    -- 2. 修正时间计算：起始绝对时间 + 相对偏移量（注意单位是纳秒）
    -- 假设你想查 UI 时间 1.92 秒 到 3.836 秒 的数据
    -- 1.92 秒 = 1920000000 纳秒
GROUP BY name
ORDER BY total_duration_us DESC;
```

2.1.4 查询某一类kernel

2.1.4.1 elementwise

查询片选时间内的包含有elementwise字段的'kernel', 'gpu_memcpy', 'gpu_memset'类别，下面的示例中规定了片选区域的时间（如果不做时间限制，会直接导出整个json文件的搜索结果）

代码块

```
SELECT
    name,
    COUNT(*) AS record_count,
    SUM(dur) / 1000000.0 AS total_duration_ms,
    SUM(dur) / COUNT(*) / 1000000.0 AS avg_duration_ms
FROM slice, trace_bounds  -- 1. 关联 trace_bounds 表获取起始时间
WHERE
    category IN('kernel', 'gpu_memcpy', 'gpu_memset')
    AND name LIKE '%elementwise%'
    -- 2. 修正时间计算：起始绝对时间 + 相对偏移量（注意单位是纳秒）
    -- 假设你想查 UI 时间 1.92 秒 到 3.836 秒 的数据
    -- 1.92 秒 = 1920000000 纳秒
    AND ts >= trace_bounds.start_ts + 4378000000
    AND ts < trace_bounds.start_ts + 6617100000
GROUP BY name
ORDER BY total_duration_ms DESC;
```

2.1.5 查询片选时间段kernel的总耗时及占比

🥖 perfetto打开之后时间被归零，且是us，需要换算为ns

代码块

```
WITH time_window AS (
  SELECT
    453770000 AS start_offset_ns,    -- 🔹 窗口起始偏移（ns）
    502315000 AS end_offset_ns       -- 🔹 窗口结束偏移（ns）
)
SELECT
    name,
    COUNT(*) AS record_count,
    SUM(dur) / 1000.0 AS total_duration_us,
    SUM(dur) / COUNT(*) / 1000.0 AS avg_duration_us,

    -- 🔹 占比1：相对占比（不含空泡）
    ROUND(SUM(dur) * 100.0 / SUM(SUM(dur)) OVER (), 2) AS kernel_relative_pct,

    -- 🔹 占比2：绝对占比（含空泡），分母 = end - start 自动计算
    ROUND(SUM(dur) * 100.0 / (tw.end_offset_ns - tw.start_offset_ns), 2) AS
window_absolute_pct

FROM slice, trace_bounds, time_window tw
WHERE
    category IN('kernel', 'gpu_memcpy', 'gpu_memset')
    AND ts >= trace_bounds.start_ts + tw.start_offset_ns
```

```
    AND ts < trace_bounds.start_ts + tw.end_offset_ns
GROUP BY name
ORDER BY total_duration_us DESC;
```

2.1.6 查询所有时间段的kernel的总耗时和占比

代码块

```
SELECT
    name,
    COUNT(*) AS record_count,
    SUM(dur) / 1000.0 AS total_duration_us,
    SUM(dur) / COUNT(*) / 1000.0 AS avg_duration_us,

    -- 🔹 占比1：相对占比（不含空泡）
    -- 含义：该 Kernel 耗时占所有 (kernel+memcpy+memset) 总耗时的比例
    ROUND(SUM(dur) * 100.0 / SUM(SUM(dur)) OVER (), 2) AS kernel_relative_pct,

    -- 🔹 占比2：绝对占比（含空泡）
    -- 修改点：分母改为整个 Trace 的总时长 (end_ts - start_ts)
    ROUND(SUM(dur) * 100.0 / (SELECT end_ts - start_ts FROM trace_bounds), 2)
AS trace_absolute_pct

FROM slice
WHERE
    -- 只保留类别过滤，去掉了时间范围过滤
    category IN('kernel', 'gpu_memcpy', 'gpu_memset')
GROUP BY name
ORDER BY total_duration_us DESC;
```

3. py过滤脚本

3.1 计算某一类kernel总耗时

⚽ 该脚本可以从json文件建立cpu op到gpu kernel的索引，筛选出前向反向的总耗时

代码块

```
# 过滤 conv 的总耗时
python kernel_op_analyzer.py bw_nhwc_20260311.json --find-op convolution
```

kernel_op_analyzer.py

3.2 prof中kernel归类

- 将prof中所有的kernel进行归类，此脚本的归类逻辑参考凯哥的excel公式（后续可能会修改判断逻辑）：

gemm对比.xlsx

- 将上面的excel公式转为py文件

extract_kernel_to_csv.py

DCU

代码块

```
=IFS(OR(ISNUMBER(SEARCH({"reduce","sampler"},A2))),"其他",ISNUMBER(SEARCH("flash",A2)),"attention",OR(ISNUMBER(SEARCH({"conv","ncxhw","cxhw","igemm","implicitgemm","MiOpen","winograd","depthwise","batchnorm","wrw"},A2))),"conv_bn",ISNUMBER(SEARCH("Cijk",A2)),"gemm",ISNUMBER(SEARCH("elementwise_kernel",A2)),"访存",ISNUMBER(SEARCH("Memcpy",A2)),"拷贝",TRUE,"其他")
```

NV

```
=IFS(OR(ISNUMBER(SEARCH({"flash","fmha","sdpa","attn"},A2))),"attention",OR(ISNUMBER(SEARCH({"implicitgemm","conv","cudnn","xmma_fprop","batchnorm"},A2))),"conv_bn",OR(ISNUMBER(SEARCH({"cijk","gemm","cublas","cutlass","nvjet","splitkreduce","wgmma"},A2))),"gemm",OR(ISNUMBER(SEARCH({"elementwise_kernel","unrolled_elementwise","vectorized_elementwise","direct_copy_kernel","binaryfunctor","fillfunctor","scatter_gather","index_elementwise","catarray","bfloat16_copy"},A2))),"访存",OR(ISNUMBER(SEARCH({"memcpy","memset"},A2))),"拷贝",TRUE,"其他")
```

3.3 计算端到端的耗时-博世